As a simple example, imagine the `CurrencyConverter` that you've been testing in this app uses `IHttpClientFactory` to call a third-party API to retrieve the latest exchange rates. You don't want to hit that API repeatedly in your integration tests, so you want to replace the `CurrencyConverter` with your own `StubCurrencyConverter`.

The first step is to ensure that the service `CurrencyConverter` implements an interface—`ICurrencyConverter` for example—and that your app uses this interface throughout, not the implementation. For our simple example, the interface would probably look like the following:

```
public interface ICurrencyConverter
{
    decimal ConvertToGbp(decimal value, decimal rate, int dps);
}
```

You would register your real `CurrencyConverter` service in Program.cs using

```
builder.Services.AddScoped<ICurrencyConverter, CurrencyConverter>();
```

Now that your application depends on `CurrencyConverter` only indirectly, you can provide an alternative implementation in your tests.

**TIP** Using an interface decouples your application services from a specific implementation, allowing you to substitute alternative implementations. This is a key practice for making classes testable.

We'll create a simple alternative implementation of `ICurrencyConverter` for our tests that always returns the same value, `3`. It's obviously not terribly useful as an actual converter, but that's